Use of AI assistants. Problems 4.1 to 4.4 are No AI. Problem 4.5 marks **each of its parts separately**, under the rules printed with it. Answers go in **one Racket file**, built from the template: code as code, traces and explanations in comments. At a defense you may be asked to write or trace any function from this sheet on paper.

Problem 4.1 (Loops without recursion). No AI Write the three functions below with the **`for`** forms of the lecture and **no explicit recursion**. The lists in (b) have no repeated elements.

(a) A weather station logs one temperature a day. `warm-days` lists the days, counted from 1, on which the temperature was above a limit, each with its temperature.

```
(warm-days 20 '(18 23 21 17 25)) ; '((2 23) (3 21) (5 25))
(warm-days 30 '(18 23))          ; '()
```

(b) `pairs-with-sum` lists the pairs of two different elements of a list that add up to a given number, each pair once, the smaller element first.

```
(pairs-with-sum 10 '(3 7 5 2 8))    ; '((3 7) (2 8))
(pairs-with-sum 9 '(4 5 1 8 6 3))   ; '((4 5) (1 8) (3 6))
(pairs-with-sum 6 '(1 2 3))         ; '()
```

(c) A playlist is a non-empty list of tracks, each a title and a length in seconds. `longest-track` is the longest one, the first of them on a tie.

```
(longest-track '(("a" 200) ("b" 320) ("c" 250))) ; '("b" 320)
(longest-track '(("x" 100) ("y" 100)))           ; '("x" 100)
```

Problem 4.2 (On demand). No AI Consider a sapling that grows by a fifth every year. Its heights, year by year, form a sequence with no end. In the functions below, assume that the starting height is positive, the growth factor is greater than one, and the target is finite.

(a) `heights`: the stream of the heights, from a starting height and a growth factor.

```
(for/list ([h (heights 50 6/5)] [i 4]) h) ; '(50 60 72 432/5)
(stream-first (stream-rest (heights 1 2)))  ; 2
```

(b) `make-next-height`: the same heights as a generator, one per call.

```
(define next-height (make-next-height 50 6/5))
(next-height) ; 50
(next-height) ; 60
(next-height) ; 72
```

(c) `years-until`: the first year in which a stream of heights exceeds a target, counting the start as year 0, **by explicit recursion** over the stream. Then, in one sentence, explain why a stream remains reusable after this function reads it, while a generator does not.

```
(years-until 100 (heights 50 6/5)) ; 4
(years-until 40 (heights 50 6/5))  ; 0
(years-until 1000 (heights 1 2))   ; 10
```

Problem 4.3 (A datatype of your own). No AI Consider a flight, with a departure airport, an arrival airport and a duration in minutes. The `struct` and a schedule of four flights are given in the template, with `route`, which names the three airports of a connection.

```
(struct flight (from to minutes))
```

(a) `airborne`: the total minutes in the air over a list of flights.

```
(airborne flights) ; 400
(airborne '())     ; 0
```

(b) `delayed`: a flight with its duration increased by a given non-negative delay, in minutes. The original flight remains unchanged.

```
(flight-minutes (delayed (first flights) 20)) ; 115
(flight-minutes (first flights))              ; 95
```

(c) `connections`: every pair of a flight and a flight that departs where the first arrives, as a list of two flights. The order of the resulting pairs is not assessed.

```
(length (connections flights))     ; 5
(map route (connections flights))
; '(("KZN" "SVO" "LED") ("KZN" "SVO" "KZN") ("SVO" "LED" "KZN")
;   ("LED" "KZN" "SVO") ("SVO" "KZN" "SVO"))
```

Problem 4.4 (Two ways to say it). No AI Consider a list of fares and two functions over it, one written with a `for` form and one with `foldl`:

```
(define fares '(120 350 90 400))

(define (discounted fares)
  (for/list ([f fares] #:when (>= f 100))
    (* f 9/10)))

(define (dearest fares)
  (foldl (lambda (f m) (max f m)) 0 fares))
```

(a) Write `discounted2`, the same function with `map` and `filter`, and `dearest2`, the same function with `for/fold`. Check all four on `fares`.

(b) In one sentence each: which version of `discounted` reads better, and which version of `dearest`, and why.

(c) The version with `map` and `filter` walks the list twice and builds one list in between. How many lists does `discounted` build? Then apply the lecture's incremental test to the computation described by each function: could a stream version produce results without reaching the end of `fares`? Answer for `discounted` and `dearest` in one sentence each, with the reason. The definitions above still produce an ordinary list and a number, so their actual inputs must be finite.

Problem 4.5 (A fold with two accumulators, audited). Consider `extremes`, the lowest and the highest of a non-empty list of readings, with two accumulators:

```
(define (extremes xs)
  (for/fold ([lo (first xs)]
             [hi (first xs)])
            ([x (rest xs)])
    (values (min lo x) (max hi x))))
```

```
(extremes '(4 9 2)) ; 2
                    ; 9
```

Someone points out that the previous sheet did everything with `foldl`, and asks for `extremes` that way.

(a) AI allowed Ask an assistant to rewrite it with a single `foldl`, and record what it produces. Any follow-up you put to it belongs here too, and in the transcript. Paste the definition above, and then a prompt along these lines:

> Rewrite `extremes` in Racket with a single `foldl` and no `for` forms.

Use a **basic, free** model, not the strongest you can reach.

(b) No AI Check its answer by hand on `(extremes '(4 9 2))`: trace the fold by the two equations of `foldl`, give the value, and say how the two accumulators travel through the fold. Mind the argument order of the step.

(c) No AI Was the rewrite an improvement? In two or three sentences: what **`for/fold`** says that the `foldl` version has to encode, and which version you would keep.

Use of AI assistants. Part (a) only. If you use an assistant, submit `transcript.md` alongside your answer file: the whole exchange, your prompts and the assistant's replies, verbatim and unedited. Record its rewrite in part (a), but do **not** assume that it is correct. Parts (b) and (c) are answered in the answer file itself, in your own words. An answer to (b) with no trace shown scores nothing. If the recorded rewrite stops with an error, keep its definition verbatim and comment out only its example call after recording the error in (b), so that the rest of the file still runs. If you use no assistant, do the rewrite yourself in (a) and audit your own attempt.